Exoplanet Intelligence System

Machine Learning-Powered Exoplanet Classification and Radius Estimation

Team Name: Nova

Team Members:

- Rajan Prasad (Team Leader)
- Aman Kumar (Team Member)

Institution: Indian Institute of Technology (IIT-BHU)

Submission Date: 24,February 2026

1. Executive Summary

The **Exoplanet Intelligence System** is a production-ready, full-stack web application designed to assist in the analysis of astronomical data from the NASA Kepler mission. Developed for the Innorave Eco-Hackathon, this system leverages advanced machine learning algorithms to address two critical tasks in exoplanet research: the classification of exoplanet candidates (Confirmed vs. False Positive) and the estimation of planetary radii.

By integrating a robust backend built with FastAPI, a dynamic React.js frontend, and a sophisticated ML pipeline utilizing XGBoost and Scikit-Learn, the system provides an intuitive interface for real-time predictions. Key achievements include achieving an F1-Score of approximately 0.95 for classification and an RMSE of roughly 0.35 Earth radii for regression tasks. The project demonstrates a seamless integration of scientific data analysis with modern software engineering practices, offering a scalable tool for astronomical research.

2. Introduction

2.1 Background

The discovery of exoplanets—planets orbiting stars outside our solar system—has been revolutionized by missions like NASA's Kepler Space Telescope. Kepler detected thousands of transiting exoplanet candidates by monitoring the brightness of stars for periodic dips caused by planets passing in front of them. However, validating these candidates requires distinguishing true planets from false positives (such as eclipsing binaries) and accurately characterizing their physical properties, such as radius.

2.2 Problem Statement

Analyzing raw Kepler data involves processing complex, high-dimensional datasets. Manual validation is time-consuming and prone to human error. Furthermore, accurately estimating planetary parameters like radius is crucial for determining habitability but is computationally intensive. There is a need for automated, high-accuracy tools that can rapidly classify candidates and estimate physical parameters to accelerate scientific discovery.

2.3 Objectives

The primary objectives of this project are:

- To develop a machine learning model capable of classifying Kepler Objects of Interest (KOIs) as either "CONFIRMED" or "FALSE POSITIVE" with high accuracy.
- To create a regression model to estimate the planetary radius (in Earth radii) based on transit and stellar properties.
- To deploy these models within a user-friendly full-stack web application that allows researchers to input feature data and receive instant predictions and visualizations.

2.4 Dataset Description

Data Source

NASA Kepler Mission - Cumulative KOI Table

(NASA Exoplanet Archive)

Dataset Size

Total Samples: ~9,000+ KOIs

Final Cleaned Dataset (after preprocessing): ~7,000+ records

Features Used: 20+ astrophysical parameters

3. Data Analysis & Preprocessing

3.1 Dataset Description

The system utilizes the NASA Kepler mission dataset, specifically the Cumulative KOI table. This dataset includes transit photometry data and stellar parameters for thousands of potential exoplanets. Key features include:

- **Transit Properties:** Period, Duration, Depth, Impact Parameter.
- **Stellar Properties:** Effective Temperature (Steff), Surface Gravity (Slogg), Metallicity (Smet), Radius (Srad).
- **Derived Parameters:** Planetary Radius (Prad), Equilibrium Temperature (Teq), Insolation Flux (Insol).

3.2 Data Cleaning and Preprocessing

Raw astronomical data often contains noise, missing values, and outliers. The following preprocessing steps were implemented:

1. **Handling Missing Values:** Missing numerical values were imputed using the median strategy to maintain distribution robustness.
2. **Outlier Detection:** Statistical methods (IQR) were used to identify and cap extreme outliers in features like planetary radius to prevent model skew.
3. **Encoding:** Categorical target variables (CONFIRMED, FALSE POSITIVE) were label-encoded for classification tasks.
4. **Scaling:** Features were standardized using `StandardScaler` to ensure zero mean and unit variance, crucial for gradient-based optimization algorithms.

3.3 Feature Selection

To avoid the curse of dimensionality and improve model inference speed, feature importance analysis was conducted using Random Forest feature importances. The top features selected for the models include:

koi_prad (Planetary Radius)

koi_depth (Transit Depth)

koi_period (Orbital Period)

koi_steff (Stellar Effective Temperature)

koi_model_snr (Signal-to-Noise Ratio)

3.4 Target Leakage Prevention

To prevent target leakage, the following precautions were taken:

1. Features directly derived from target (e.g., confirmed planetary radius when predicting radius) were removed.
2. Post-validation parameters not available during inference were excluded.
3. Train-test split performed before scaling to prevent data leakage.
4. Pipelines used to ensure transformation consistency.
5. This ensures real-world model reliability and prevents inflated accuracy.

4. Methodology

4.1 Classification Approach

The classification task distinguishes between confirmed exoplanets and false positives. We utilized **XGBoost Classifier** due to its efficiency with tabular data and ability to handle non- linear relationships. The model was trained to optimize the F1-score, balancing precision and recall, which is critical in scientific discovery where missing a true planet (False Negative) is as detrimental as validating a false one.

4.2 Regression Approach

For estimating planetary radius, an **XGBoost Regressor** was employed. The model predicts the radius in units of Earth radii. Performance was evaluated using Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE). 95% Confidence Intervals were calculated to provide uncertainty estimates for the predictions, enhancing scientific reliability.

5. Technical Architecture

The Exoplanet Intelligence System follows a modern, decoupled microservices-ready architecture, ensuring scalability and maintainability.

[Architecture Diagram Representation]

React Frontend <--> FastAPI Backend <--> ML Inference Engine <--> PostgreSQL Database

5.1 Frontend Stack

- **React 18:** Provides a reactive, component-based UI.
- **Vite:** Ensures fast build times and hot module replacement.
- **TailwindCSS:** Utilized for rapid, utility-first styling.
- **Recharts:** Implements interactive data visualizations for model outputs.

5.2 Backend Stack

- **FastAPI:** A high-performance web framework for building APIs with Python 3.9+. It provides automatic Swagger documentation.
- **Pydantic:** Enforces strict data validation for API requests, ensuring reliable model inputs.
- **PostgreSQL (Supabase):** Stores prediction history and user feedback.

5.3 ML Pipeline

The ML pipeline is built using **scikit-learn** for preprocessing and pipelines, and **XGBoost** for the core algorithms. Models are serialized using **joblib** for efficient loading during inference.

6. Model Performance & Results

6.1 Classification Metrics

The classification model demonstrated exceptional ability to distinguish true exoplanets from background noise.

Metric	Value	Interpretation
F1-Score	~0.95	High balance between precision and recall.
ROC-AUC	~0.98	Excellent discrimination capability.
Accuracy	~96%	Overall correctness of predictions.

6.2 Regression Metrics

The regression model accurately predicted planetary radii, providing essential data for habitability analysis.

Metric	Value	Unit
RMSE	~0.35	Earth Radii
MAE	~0.28	Earth Radii
R-Squared	0.92	-

7. Features & Implementation

7.1 Web Interface

The application provides a seamless user experience. The "Classification" and "Regression" tabs allow users to toggle between tasks. The input forms feature real-time validation to ensure parameters like *Stellar Temperature* or *Orbital Period* fall within physically possible ranges.

7.2 API Design

The RESTful API provides endpoints for prediction and health checks. Below is the structure of a classification request:

```
POST /api/predict/classification
{
  "features": {
    "koi_prad": 2.5,
    "koi_depth": 150,
    "koi_period": 45.3,
    "koi_steff": 5800,
    "koi_model_snr": 25.0
  }
}
```

8. Project Folder Structure

```
exoplanet-intelligence-system/
│
├── frontend/              # React.js frontend
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   └── utils/
│   ├── package.json
│   └── vite.config.js
│
├── backend/               # FastAPI backend
│   ├── app/
│   │   ├── main.py
│   │   ├── routes.py
│   │   ├── schemas.py
│   │   ├── database.py
│   │   ├── crud.py
│   │   ├── model_loader.py
│   │   └── utils.py
│   ├── models/
│   └── requirements.txt
│
├── ml/                    # ML pipeline
│   ├── download_data.py
│   ├── preprocessing.py
│   ├── feature_selection.py
│   ├── train_classification.py
│   └── train_regression.py
│
├── database/
│   ├── schema.sql
│   └── seed_data.sql
│
├── docs/
│   ├── api_docs.md
│   └── deployment_guide.md
│
└── README.md
```

9. Deployment & Usage

9.1 Deployment Architecture

The system is designed for cloud deployment. The frontend is optimized for static hosting platforms like Vercel, while the backend is container-ready for platforms like Render or Railway.

9.2 Installation Instructions

To set up the environment locally:

11. Deployment

Frontend:

- **Vercel (Static Hosting)**

Backend:

- **Render / Railway**
- **Docker-ready**

Database:

- **PostgreSQL (Supabase)**

10. Conclusion

The Exoplanet Intelligence System successfully demonstrates the application of machine learning to astrophysics. By automating the classification and characterization of exoplanets, this tool reduces the manual workload on astronomers and provides rapid insights into vast datasets.

Scientific Impact: The high accuracy (~96%) suggests that ML models can effectively replicate human expert analysis for candidate vetting.

Future Improvements: Future iterations will include deep learning models (CNNs) to analyze raw light curve images directly, further enhancing predictive capabilities.

11. References

- **GitHub Repository:** https://github.com/AmanKumar1612/Exoplanet-Intelligence
- **Data Source:** NASAExoplanet Archive - Kepler Mission Data.
- **Tech Stack:** React,FastAPI, Scikit-Learn, XGBoost, PostgreSQL.
- **Live Application:** http://exoplanet-intelligence.vercel.app/